

SD Dictionaries - Structure

The dictionary file: every record type, and every field of each one.

A dictionary is a file that describes a data file. It holds the field names, the field numbers they point at, the format in which they print, and the expressions that compute values that are not stored. Every `LIST`, `SELECT`, `SORT` and `COUNT` command reads the dictionary to find out what to read and how to display it. This page is about the record types in a dictionary, their fields, and how they relate to the data file.

SD folds case, so a field name may be typed in either case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

Every record on this page was read from the system dictionaries on SD Core for Windows W1.0-0. The `voc.dic` dictionary — the dictionary of the VOC — is a stock file whose records illustrate every type.

What a dictionary is

Every data file has a dictionary. The VOC record for a file of type `F` carries two paths: field 2 is the data path, field 3 is the dictionary path. If field 3 is empty, the file has no dictionary and cannot be used by the query processor — `LIST` on it fails with *"no dictionary"*.

The dictionary is itself a hashed file. Its operating-system name is the file name with `.dic` appended — `bp.dic`, `voc.dic`, `stock.dic`. The `dict` verb opens it: `dict file.name` is the same as opening `DICT file.name` from BASIC.

The system file `dict.dic` in `@SDSYS` is the dictionary *of* the dictionary — its own dictionary, which describes the fields a dictionary record has. It is what `dict dict.dic` reads.

The dictionary record types

Field 1, character 1, is the type. The rest of field 1 is free text and is ignored, so a record whose field 1 reads `D Data for stock qty` is type `D` and the rest is a comment.

TYPE	NAME	WHAT IT DOES
D	direct	references a field number in the data record
I	indirect	an expression that returns a value — a virtual field

TYPE	NAME	WHAT IT DOES
C	calculated	like I but compiled with the BASIC compiler and returning via @ANS
A	A-type	Pick-style data definition, with a different field layout
S	S-type	identical to A in SD — kept for compatibility
L	link	used only in query processor commands, to join to another file
PH	phrase	a stored fragment substituted into a query command line
X	text	miscellaneous data — not a field definition

A and S are the Pick layout, not the SD layout. They carry the same information as D but in different fields — field 3 holds the display name, field 7 the conversion, field 8 the correlative, field 9 the justification, field 10 the width. SD supports them because applications imported from Pick use them. New dictionaries should use D and I.

The D record — a direct field reference

```
001  D
002  2
003
004  F2
005  5L
006  S
```

This is F2 from voc.dic. Field 1 is D, field 2 is the field number in the data record, and the remaining fields control how it appears in a query.

FIELD	NAME	WHAT IT HOLDS
1	type	D, optionally followed by a comment
2	location	the field number in the data record
3	conversion	a conversion code, or empty
4	display name	the column heading; \ for none; 'R' prefix right-justifies
5	format	width and justification — see below
6	S/M	S for single-valued, M for multivalued

FIELD	NAME	WHAT IT HOLDS
7	association	a name grouping multivalued fields, or empty
8	user	free for the application — SD does not read it

PSEUDO-FIELDS

Field 2 is not always a literal field number. Three values have special meaning:

LOCATION	WHAT IT READS
0	the record id (@ID)
9998	@NI — the number of items in a multivalued field set
9999	the record length in bytes (@NB)

A dictionary record with location 0 is how the record id appears in a query. The @ID record in voc.dic has location 0 and display name @ID.

THE FORMAT FIELD

Field 5 is the format specification — the column width and justification the query processor uses when it prints the field. It is a compact syntax:

SYNTAX	MEANING
<i>n</i> L	left-justified, <i>n</i> characters wide
<i>n</i> R	right-justified, <i>n</i> characters wide
<i>n</i> T	text — left-justified, wrapping at word boundaries
<i>n</i> C	centred, <i>n</i> characters wide
<i>n</i> L <i>d</i>	left-justified with <i>d</i> decimal places (numeric)
<i>n</i> R <i>d</i>	right-justified with <i>d</i> decimal places (numeric)

5L means five characters, left-justified. 20L means twenty characters, left-justified. 10R2 means ten characters, right-justified, two decimal places. The format field is also what FMT() uses in BASIC.

SINGLE AND MULTIVALUED

Field 6 is S or M. A single-valued field holds one value per record; a multivalued field holds many, separated by value marks. The query processor prints each value of a multivalued field on its own line unless the field is in an **association**.

ASSOCIATIONS

Field 7 names an association — a group of multivalued fields whose values are parallel. If `qty` and `price` are both multivalued and both carry association `LINE.ITEM`, the query processor prints them side by side: `qty` value 1 next to `price` value 1, and so on.

An association is a name, not a structure. The fields that share one are linked by having the same name in field 7, not by anything stored separately. There is no association record.

The I record — an indirect field

```
001  I
002  IF DATA.NAME # '' THEN SUBR("!FType",DATA.NAME) ELSE ''
003
004  FType
005  8L
006  S
```

This is `FType` from `voc.dic`. Field 1 is `I`, field 2 is an expression in SD BASIC syntax, and the remaining fields are the same as for a D-record. The expression is compiled to object code, which is stored in field 16 onward, and executed when the query processor needs the value.

WHAT AN I-TYPE EXPRESSION CAN USE

An I-type expression is a single BASIC expression — not a program. It can use:

VARIABLE	WHAT IT HOLDS
<code>@ID</code>	the record id
<code>@RECORD</code>	the entire data record as a dynamic array
<code>n</code> or <code>F n</code>	field <i>n</i> of the data record — <code>F2</code> is field 2
<code>@NI</code>	the number of values in the longest multivalued field
<code>@NB</code>	the record length in bytes
<code>@FILE.NAME</code>	the name of the file being queried
<code>@USER.NO</code>	the user number of the session
<code>@PATH</code>	the account path

It can call `SUBR()` to invoke a catalogued subroutine, use `TRANS()` to read from another file, and use any BASIC function — `LEN()`, `FIELD()`, `RAISE()`, `OCONV()`, `ICONV()`, `COUNT()`, `SUM()`, `DATE()`, `TIME()` and the rest. It cannot run loops or call statements that take a variable number of arguments.

COMPILATION

An I-type is compiled the first time it is used after it is written or edited. The compiled object is stored in field 16 onward of the same dictionary record, and a stamp in field 15 tells the query processor whether the object is current. **Editing field 2 invalidates the object.** The next query recompiles it. If the expression has a syntax error, the query reports it and the field appears empty.

THE FIELD 15 STAMP

Field 15 is the system information field. SD stores a hash of the expression source there when it compiles the I-type. When a query opens the dictionary, it compares the hash to the source — if they differ, the I-type is recompiled. This is why field 15 has a large number in a stock I-type record and is empty in a D-type record.

The C record — a calculated field

A C-type is like an I-type but is compiled with the full BASIC compiler rather than the I-type expression compiler. The expression in field 2 is a complete BASIC program that returns its result in `@ANS`. C-types are rare in stock dictionaries — I-types cover most computed fields — but they exist for compatibility with applications that use them.

The A record — a Pick-style field

An A-type (or S-type) uses a different field layout:

FIELD	NAME	WHAT IT HOLDS
1	type	<code>A</code> or <code>S</code> , optionally followed by a comment
2	location	the field number in the data record
3	display name	the column heading
4	association	a name grouping multivalued fields
5 - 6		not used
7	conversion	a conversion code
8	correlative	a correlative code — like a mini I-type

FIELD	NAME	WHAT IT HOLDS
9	justify	L or R
10	width	column width

The correlative in field 8 is a Pick-style expression — a short code that transforms the field value before display. SD compiles correlatives the same way it compiles I-types, storing the object in field 16 onward.

The PH record — a phrase

```
001  PH
002  GRAND.TOTAL  "' L ' "
```

A phrase is a stored fragment substituted into a query command line wherever its name appears. PH records in a dictionary work the same way as PH records in the VOC — the difference is that a dictionary phrase is scoped to the file whose dictionary holds it.

The L record — a link

An L-type is used only in query processor commands. It names another file to join to, so that fields from both files can appear in one report. An L-record carries the file name and the field to match on, and the query processor reads from the linked file for each record it lists.

The X record — text

An X-record in a dictionary is miscellaneous data — not a field definition. It can hold anything the application wants to store by name in the dictionary.

Continued in

SD Dictionaries - Using and Creating — how the query processor uses a dictionary, the system dictionaries, and creating one.

